Can a 100M-Parameter Transformer Learn Simple Python Programming Tasks?

George Yuan
1012152224, zhijie.yuan@mail.utoronto.ca
GitHub: https://github.com/ShiinaMahiruOvO/aps360-small-code-lm

Introduction

Modern code models usually contain billions of parameters, which makes it unclear how much programming ability can be learned by a much smaller model trained with a course-project budget. Our project asks a direct question: *can a decoder-only Transformer with approximately 100 million parameters, pretrained from scratch on Python code, learn to solve simple programming tasks from natural-language specifications?*

We define a simple task as one self-contained Python function requiring no external packages and no advanced topics such as dynamic programming, graph algorithms, or backtracking. The evaluation covers three increasingly structured groups: basic string/list/dictionary manipulation; linear scans, counting, and elementary sorting or search; and introductory data-structure operations, with binary search trees (BSTs) as the main structured case study. Deep learning is suitable because both specifications and valid programs are variable-length sequences and many semantically equivalent solutions exist, making fixed templates too brittle. Functional correctness is the primary outcome: a generated program passes only if it satisfies every hidden unit test. We will report pass@1 overall and by task group, with pass@5 as a secondary measure.

This is an individual project. I will implement and document the data pipeline, model architecture, training procedures, execution-based evaluation, and final analysis. Configuration files, fixed random seeds, and version-controlled scripts will be used to keep the experiments reproducible.

Illustration

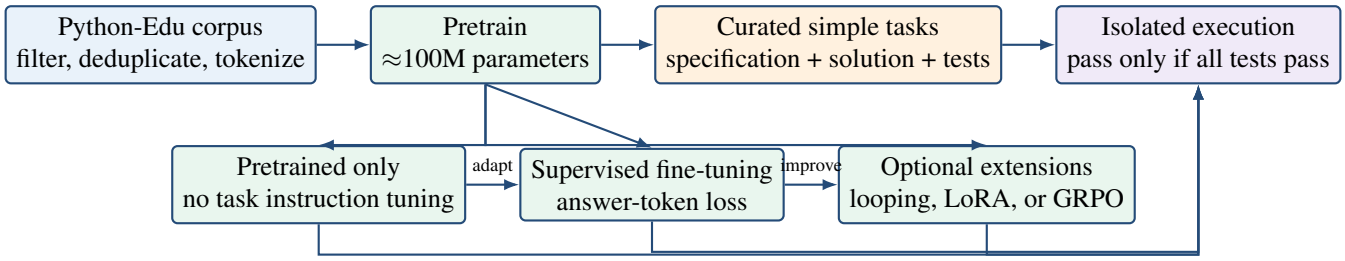


Figure 1: Project pipeline. The main experiment measures how much simple programming ability is acquired through pretraining and supervised task adaptation; architectural and reinforcement-learning changes are secondary extensions.

Background & Related Work

The Transformer established attention-only sequence modelling and is the basis of our decoder [8]; RoPE provides relative position information for its attention layers [7]. Codex and HumanEval popularized execution-based functional correctness for code generation [3], while MBPP introduced a crowd-sourced benchmark of short Python problems that closely matches our intended difficulty [1]. SmolLM-Corpus provides the educational Python source used for pretraining [2]. Universal Transformers show that repeatedly applying shared layers can improve algorithmic processing [4], and LoRA provides a parameter-efficient way to specialize shared weights [5]. These last two methods are optional extensions; the project's main contribution is a controlled study of what a from-scratch 100M-parameter model can learn.

Data Processing

For pretraining, we use the Python-Edu subset of the public SmolLM-Corpus [2], which contains Python files selected for educational quality. Our pipeline removes empty or extremely short files, generated/minified content, malformed or non-Python records, and exact duplicates; it also rejects samples that fail Python syntax parsing. Repositories, rather than individual files, are assigned to train/validation splits to reduce leakage. We train an 8,192-token byte-pair tokenizer and serialize token IDs into memory-mapped shards. The current processed corpus contains approximately 1.03M training documents (498M tokens) and 55M validation tokens. Scripts and configuration files record every filter, seed, split, and tokenizer setting.

For supervised fine-tuning, we will combine easy examples filtered from KodCode-Light-RL-10K [6] with tasks generated by us. Automated filters require a single self-contained Python function, a reference solution of at most 30 logical lines, no external dependencies, and tests that finish within one second. We remove incomplete statements, answer leakage, syntax failures, and exact or normalized duplicates, then manually

review category and difficulty. Our existing generated subset contains 552 training examples and covers 17 BST operation contracts, including search, insertion, predecessor/successor, range count, range sum, and path queries. We will add comparably simple string, collection, counting, sorting, and searching problems.

Validation and evaluation tasks are split by normalized specification before prompt templates are rendered. We will use an easy, manually verified subset of MBPP plus held-out generated tasks, and remove statement or solution overlaps with the fine-tuning set. Every record contains a prompt, trusted reference implementation, and deterministic tests; all trusted solutions must pass before inclusion. Candidate programs run in time-limited, network-disabled subprocesses. This processing contributes more than simply downloading an existing benchmark and makes task difficulty and leakage checks reproducible.

Architecture

The main model is a decoder-only Transformer with an 8,192-token vocabulary, context length 1,024, hidden width 768, 12 layers, 12 attention heads, RMSNorm, RoPE, and SwiGLU ($\approx$ 97.5M parameters). It is first trained with next-token prediction on Python-Edu and then with answer-token-only supervised loss on instruction–solution pairs. We will generate code autoregressively and stop at an end-of-answer marker. If the standard model establishes a useful baseline, we will test a parameter-matched looped variant whose eight middle blocks are executed twice, iteration-specific rank-32 LoRA updates on its query and value projections, and execution-reward GRPO. These extensions are ablations rather than requirements for answering the main question.

Baseline Model

Our simplest baseline retrieves the solution attached to the most similar training prompt using TF–IDF cosine similarity; it requires no neural training and tests whether the benchmark can be solved by surface-level matching. A second baseline evaluates the pretrained 100M model before supervised fine-tuning. The main comparison uses the same model after supervised fine-tuning, with identical decoding and test execution. We will report pass@1 and pass@5 overall and for each task group, together with validation loss, parameter count, and training cost. We will regard the project as demonstrating useful learned ability if the fine-tuned model reaches at least 40% pass@1 overall, attempts every category successfully, and exceeds both the retrieval and pretrained-only baselines by at least 10 percentage points. Qwen2.5-Coder-0.5B may be reported as a larger external reference, but not as the primary baseline.

Ethical Considerations

Training on public code creates licensing, attribution, privacy, and memorization risks; dataset provenance is therefore documented, duplicates are removed, and model outputs will not be presented as original human work. Generated programs can be subtly incorrect or insecure, so our model is an educational prototype rather than a deployment-ready programming assistant. Untrusted completions are executed only in resource-limited subprocesses with timeouts and no network access. We will report failures and possible benchmark contamination, avoid claims beyond beginner-level Python tasks, and release model/data documentation that states these limitations.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. Smollm-corpus. Hugging Face dataset, 2024. URL https://huggingface.co/datasets/HuggingFaceTB/smollm-corpus.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[6] KodCode. Kodcode-light-rl-10k. Hugging Face dataset, 2025. URL https://huggingface.co/datasets/KodCode/KodCode-Light-RL-10K.

[7] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.